

Trabajo Práctico Obligatorio

Programación Concurrente

Simulación del Sistema de Aeropuerto "VIAJE BONITO"

Carrera: Licenciatura en Ciencias de la Computación

Nombre y Apellido: Daniel Alexis Carrasco Cifuentes

Legajo: FAI - 2840

DNI: 43 552 859

Año de cursada aprobado: 2023

Índice

1. Introducción
2. Metodología de Trabajo
3. Arquitectura del Sistema
4. Mecanismos de Sincronización
 - 4.1. Semáforos (Semaphore)
 - 4.2. Monitores (synchronized / wait / notifyAll)
 - 4.3. Locks (ReentrantLock + Condition)
 - 4.4. CyclicBarrier
 - 4.5. CountdownLatch
 - 4.6. Exchanger
 - 4.7. BlockingQueue
5. Ejecución Paso a Paso desde Main
6. Ciclo de Vida de un Pasajero
7. Manejo de Casos Extremos
8. Conclusiones

1. Introducción

El presente informe documenta el desarrollo del Trabajo Práctico Obligatorio de la materia Programación Concurrente, en el cual se simula el funcionamiento del Aeropuerto "VIAJE BONITO". El sistema modela el recorrido completo de un pasajero desde que llega al aeropuerto hasta que sube al avión, incluyendo:

- Ingreso al aeropuerto y control de apertura/cierre.
- Puesto de informes donde se deriva al pasajero a una aerolínea.
- Puesto de atención de aerolínea con control de capacidad y guardia.
- Intercambio de boleto mediante Exchanger entre pasajero y empleado.
- Transporte interno (people mover) con capacidad de 5 pasajeros.
- Terminal con free-shop (capacidad limitada, cajeros) y sala de embarque.
- Despegue del avión cuando todos los pasajeros han embarcado.

El aeropuerto atiende pasajeros de 6:00 a 22:00 horas. Cuenta con 3 terminales (A, B, C), 3 aerolíneas (Aerolíneas Argentinas, LATAM, JetSMART), 2 puestos de ingreso, 2 cajeros por terminal y un transporte interno de capacidad 5.

2. Metodología de Trabajo

Para el desarrollo de este sistema concurrente se utilizó una metodología incremental:

Análisis de la consigna: Se identificaron todos los componentes del sistema y las restricciones de sincronización requeridas por la materia: semáforos, monitores, Locks, CyclicBarrier, CountdownLatch, Exchanger y BlockingQueue.

Diseño de la arquitectura: Se modelaron las entidades como hilos independientes: Pasajero, Guardia, EmpleadoAtencion, EmpleadoInforme, EmpleadoSalon, Cajero, Conductor y Administrador.

Implementación por componentes: Cada componente se implementó de forma independiente: primero el ingreso (ControlAeropuerto), luego los puestos (PuestoInformes, PuestoAtencion), el transporte (TransporteATerminal), las terminales (FreeShop, SalaEmbarque) y finalmente la integración en Main.

Resolución de condiciones de carrera: Se identificaron y corrigieron problemas de sincronización mediante pruebas repetidas con 20 pasajeros, verificando que todos subieran al avión.

Validación: Se ejecutó el programa múltiples veces verificando que 20/20 pasajeros completaran el recorrido y que el mensaje de despegue apareciera al final.

3. Arquitectura del Sistema

El sistema está compuesto por los siguientes componentes:

3.1. Diagrama de Componentes

Estructura jerárquica del sistema:

```
Main
├── Aeropuerto
│   ├── Terminal[3] (A, B, C)
│   │   ├── FreeShop (capacidad=3)
│   │   └── SalaEmbarque
│   ├── TransporteATerminal (capacidad=5)
│   │   ├── CyclicBarrier(5)
│   │   ├── ArrayBlockingQueue(5)
│   │   ├── Semaphore mutex
│   │   └── Semaphore[] barreraTerminal[3]
│   ├── IngresoAeropuerto
│   │   ├── PuestoAtencion[3] (Aerolíneas, LATAM, JetSMART)
│   │   └── PuestoInformes
│   └── CountdownLatch(20) avionDespega
├── ControlAeropuerto (ReentrantLock + 2 Conditions)
├── AdministradorAeropuerto
├── 3 Guardia + 3 EmpleadoAtencion
├── EmpleadoInforme
├── 3 EmpleadoSalon
├── 6 Cajero (2 por cada terminal)
├── 20 Pasajero
└── Vuelo (CountDownLatch.await)
```

3.2. Tabla de Hilos

Hilo	Cantidad	Recurso Compartido	Rol
AdministradorAeropuerto	1	ControlAeropuerto	Abre y cierra el aeropuerto
Guardia	3	PuestoAtencion[0..2]	Controla capacidad de cada puesto
EmpleadoAtencion	3	PuestoAtencion[0..2]	Realiza intercambio de boletos
EmpleadoInforme	1	PuestoInformes	Deriva pasajeros a aerolíneas
EmpleadoSalon	3	Terminal.sala	Llama a embarcar por terminal
Cajero	6	Terminal.tienda	Procesa pagos del free-shop
Conductor	1	TransporteATerminal	Conduce el transporte interno
Pasajero	20	Múltiples	Recorre todo el sistema
Vuelo	1	CountDownLatch	Espera a que todos embarquen

4. Mecanismos de Sincronización

La consigna requiere el uso obligatorio de: semáforos, monitores (synchronized), Locks, CyclicBarrier, CountDownLatch, Exchanger e implementaciones de BlockingQueue. A continuación se describe dónde y por qué se utiliza cada mecanismo.

4.1. Semáforos (Semaphore)

Los semáforos se utilizan en TransporteATerminal para múltiples propósitos de control de concurrencia:

Semáforo	Tipo	Propósito
mutex	Binario (1, true)	Protege las variables compartidas pasajerosABordo[] y totalSubidos
mutexArranque	Binario (1, true)	Protege la variable boolean conductorArrancando
mutexAeropuerto	Binario (1, true)	Protege la variable boolean aeropuertoCerrado
puedeAbordar	Generico (5, true)	Limita el abordaje a 5 pasajeros antes de que la barrera se active
barreraTerminal[3]	Generico (0, true)	Permite al conductor liberar pasajeros en cada terminal
inicioUltimoViaje	Generico (0)	Sincroniza pasajeros atrapados en BrokenBarrierException

Se eligieron semáforos porque los semáforos binarios (mutex) con un solo permiso protegen variables booleanas compartidas. Mientras que los semáforos de conteo controlan capacidad y sincronización por terminal.

4.2. Monitores (synchronized / wait / notifyAll)

Los monitores se implementan mediante la palabra clave “synchronized” de Java, combinada con wait() y notifyAll() para la coordinación entre hilos. FreeShop y PuestoAtencion usan exclusivamente monitores

PuestoAtencion.java

El monitor en PuestoAtencion controla la capacidad del puesto de atención. El patrón implementado es productor-consumidor: el Guardia produce permisos de ingreso, y los pasajeros los consumen.

Variables protegidas:

- activos: cantidad de pasajeros dentro del puesto.
- esperando: pasajeros esperando permiso del guardia.
- permisosPendientes: permisos otorgados por el guardia pero aún no consumidos.

El Guardia ejecuta permitirIngreso() en un loop infinito: espera mientras no haya pasajeros esperando, el puesto esté lleno, o haya un permiso pendiente. Cuando las condiciones lo permiten, incrementar permisosPendientes y notifica.

FreeShop.java

El FreeShop utiliza synchronized + wait/notifyAll para tres funciones:

- Control de capacidad: synchronized method + timed wait. El contador 'contador' rastrea cuántos pasajeros hay dentro. Si está lleno, el pasajero espera con wait(restante) hasta que haya lugar o se agote el tiempo.
- Realizar pago: El pasajero setea pagoPendiente=true, ejecuta notifyAll() para despertar al cajero, y espera con wait(). El cajero procesa el pago (Thread.sleep(3s)), setea pagoPendiente=false, y notifica al pasajero.
- Salida FIFO: Se usa un patrón de turnos con turnoActual y esperandoTurno. Cada pasajero toma un número de turno (miTurno = esperandoTurno++), y espera con wait() mientras turnoActual != miTurno. Al ser su turno, incrementa turnoActual y ejecuta notifyAll() para despertar al siguiente.

4.3. Locks (ReentrantLock + Condition)

Los Locks de tipo ReentrantLock con Condition variables se utilizan en tres componentes donde se necesita esperar en múltiples conjuntos de espera o realizar timed waits.

ControlAeropuerto.java

Utiliza ReentrantLock con 2 Conditions: "pasajeros" y "administrador". Los pasajeros esperan en "pasajeros" hasta que el aeropuerto abra en "(while(!abierto) await())". El administrador espera 30 segundos en "administrador" (await con timeout) antes de cerrar. Se usa Lock en lugar de synchronized porque se requiere un await con timeout para el administrador.

PuestoInformes.java

Utiliza ReentrantLock con 3 Conditions: "esperandoTurno", "empleadoEspera" y "procesado". Modela un encuentro de tres partes: el pasajero siguiente espera en esperandoTurno, el pasajero actual espera en procesado, y el empleado espera en empleadoEspera. Las 3 Conditions permiten coordinar estos tres roles de forma independiente.

SalaEmbarque.java

Utiliza ReentrantLock con 2 Conditions: "pasajeroLlego" y "llamado". Implementa un patrón de generación (grupo) para boarding cíclico: el empleado espera en pasajeroLlego hasta que haya pasajeros, luego incrementa el grupo y señala a todos en llamado. Los pasajeros que llegan después quedan en la siguiente generación.

4.4. CyclicBarrier

Se utiliza en TransporteATerminal con capacidad 5 y una acción de barrera (avisarConductor). Cuando los 5 asientos del transporte se llenan, la barrera se activa automáticamente y lanza un hilo Conductor que recorre las terminales.

Se usa CyclicBarrier porque se necesita esperar a que exactamente N hilos lleguen a un punto de sincronización antes de proceder. El CyclicBarrier es reutilizable por reset() para manejar viajes parciales cuando el aeropuerto cierra.

4.5. CountdownLatch

Se utilizan dos CountdownLatch con propósitos diferentes:

avionDespega (Aeropuerto.java)

Inicializado con cantidadPasajeros (20). Cada pasajero hace countdown() después de embarcar. Un hilo "Vuelo" hace await() y cuando todos han embarcado, imprime "EL AVION DESPEGA CON TODOS LOS PASAJEROS A BORDO". Este es el uso principal y significativo del CountdownLatch: representa un evento irreversible que ocurre cuando N eventos se completan.

finPasajeros (TransporteATerminal.java)

También inicializado con la cantidad total de pasajeros. Se utiliza internamente para detectar cuando no quedan más pasajeros por llegar (getCount() == 0). Esto permite decidir si un viaje parcial debe partir cuando la barrera se resetea por cierre del aeropuerto.

4.6. Exchanger

Se utiliza en PuestoAtencion para el intercambio simétrico de datos entre el pasajero y el empleado de atención. El pasajero envía su boleto de avión y recibe un boleto con la terminal y puesto de embarque asignados.

Se usa Exchanger porque es el mecanismo ideal para un intercambio 1-a-1 donde ambos hilos intercambian datos simultáneamente. Un boolean "intercambioEnCurso" serializa los intercambios para evitar colisiones, ya que el Exchanger solo soporta dos hilos a la vez.

4.7. BlockingQueue

Se utiliza ArrayBlockingQueue<Boolean> de capacidad 5 en TransporteATerminal para modelar los asientos físicos del transporte interno.

Operaciones:

- put(true): el pasajero se sienta. Bloquea si los 5 asientos están ocupados.
- poll(): el pasajero se baja en su terminal. Libera un asiento.

Se usa BlockingQueue porque proporciona capacidad acotada con bloqueo automático. Cuando el transporte está lleno, los pasajeros adicionales se bloquean naturalmente en put() hasta que alguien haga poll(). Es más limpio que implementar capacity control manual con semáforos o wait/notify.

4.8. Resumen de Mecanismos

Mecanismo	Archivo	Uso
Semaphore	TransporteATerminal	Mutex, mutexArranque, mutexAeropuerto, barreraTerminal, puedeAbordar, inicioUltimoViaje
synchronized	PuestoAtencion, FreeShop	Control de capacidad, pagos, salida FIFO, guardia

ReentrantLock + Condition	ControlAeropuerto, PuestoInformes, SalaEmbarque	Apertura/cierre, informes, embarque
CyclicBarrier	TransporteATerminal	5 pasajeros llenan el transporte, avisa al conductor
CountDownLatch	Aeropuerto, TransporteATerminal	Despegue del avión, viajes parciales
Exchanger	PuestoAtencion	Intercambio boleto entre pasajero y empleado
BlockingQueue	TransporteATerminal	Asientos del transporte (capacidad 5)

5. Ejecución Paso a Paso desde Main

A continuación se describe el flujo completo de ejecución del programa, desde Main hasta el despegue del avión.

5.1. Inicialización

1. Se crea un Aeropuerto con 3 terminales (A, B, C), un TransporteATerminal (capacidad 5, 3 terminales), un IngresoAeropuerto (2 puestos) con 3 PuestoAtencion y 1 PuestoInformes, y un CountdownLatch(20).
2. Se crea un ControlAeropuerto que referencia al Aeropuerto.
3. Se inicia el hilo AdministradorAeropuerto: llama abrirAeropuerto() (abierto=true, señala a los pasajeros que esperan) y luego cerrarAeropuerto() (espera 30s, cierra el ingreso, notifica al transporte).
4. Se inician 3 Guardias y 3 EmpleadoAtencion, uno por cada PuestoAtencion.
5. Se inicia 1 EmpleadoInforme para el PuestoInformes.
6. Se inician 3 EmpleadoSalon, uno por cada terminal.
7. Se inician 6 Cajeros, dos por cada terminal.
8. Se inician 20 hilos Pasajero (índices pares con comprar = true, impares con comprar = false).
9. Se inicia el hilo Vuelo que hace await() en el CountdownLatch.

5.2. Flujo del Administrador

El hilo Administrador ejecuta dos operaciones secuenciales:

1. abrirAeropuerto(): adquiere el lock, setea abierto=true, y ejecuta pasajeros.signalAll() para despertar a todos los pasajeros que estaban esperando a que abra el aeropuerto.
2. cerrarAeropuerto(): ejecuta administrador.await(30, TimeUnit.SECONDS) — espera 30 segundos (simulando las horas de operación 6:00-22:00). Luego setea abierto=false y llama a

transporte.notificarAeropuertoCerrado()). Este método adquiere mutexAeropuerto, setea aeropuertoCerrado=true, y ejecuta barrera.reset() directamente para romper la barrera.

5.3. Flujo de los Guardias y Empleados de Atención

Cada Guardia ejecuta un loop infinito llamando puesto.permitirIngreso(). Espera mientras: no haya pasajeros esperando, el puesto esté lleno, o haya un permiso sin consumir. Cuando hay espacio, otorga un permiso y notifica.

Cada EmpleadoAtencion ejecuta un loop infinito llamando puesto.intercambio(). Espera a que un pasajero solicite un intercambio (intercambioEnCurso == true), genera un boleto aleatorio (terminal A/B/C y puesto de embarque), y lo intercambia mediante Exchanger.

5.4. Flujo del Empleado de Informes

El EmpleadoInforme ejecuta un loop infinito llamando puesto.atenderPasajero(). Espera a que un pasajero llegue (empleadoEspera.await()), selecciona aleatoriamente uno de los 3 puestos de atención, y asigna al pasajero a ese puesto.

6. Ciclo de Vida de un Pasajero

Cada pasajero atraviesa las siguientes fases:

Fase 1: Ingreso al Aeropuerto

El pasajero llama a control.entrarAlAeropuerto(). Adquiere el ReentrantLock de ControlAeropuerto y espera en while(!abierto) pasajeros.await(). Se despierta cuando el administrador abre el aeropuerto. Retorna true indicando que pudo ingresar.

Fase 2: Puesto de Informes

El pasajero llama a informe.llegarAlInforme(). Adquiere el lock de PuestoInformes. Si otro pasajero está siendo atendido (ocupado == true), espera en esperandoTurno. Una vez libre, señala al empleado y espera en procesado hasta que le asignen un puesto. El empleado asigna aleatoriamente un PuestoAtencion y el pasajero lo recibe.

Fase 3: Entrada al Puesto de Atención

El pasajero llama a puesto.puedeEntrarPuesto(). Incrementa “esperando” y espera en while(permisosPendientes == 0) wait(). El Guardia ve que hay alguien esperando y hay espacio, y otorga un permiso. El pasajero consume el permiso, incrementa “activos” y decrementa “esperando”.

Fase 4: Intercambio de Boleto

El pasajero llama a puesto.realizarIntercambio(boletoAvion). Espera mientras otro intercambio está en curso, luego setea intercambioEnCurso=true y llama a exchanger.exchange(boletoAvion). Se bloquea hasta que el empleado también llame a exchanger.exchange(boletoTerminal). El Exchanger realiza el intercambio: el pasajero recibe su destino (ej: ["B", "12"]).

Fase 5: Salida del Puesto

El pasajero llama a `puesto.salirPuesto()`. Decrementa “activos” y notifica, permitiendo que el Guardia pueda dar paso a otro pasajero.

Fase 6: Transporte al Terminal

El pasajero llama a `transporte.subirATransporte(terminal)`. Adquiere un permiso de `puedeAbordar`, hace `asientos.put(true)` (bloquea si no hay lugar), actualiza contadores con `mutex`, hace `finPasajeros.countDown()`, y espera en `barrera.await()`. Cuando se completan 5 pasajeros, la barrera activa al Conductor. El Conductor recorre las terminales A, B y C, liberando pasajeros en cada una. El pasajero se baja haciendo `bajarDelTransporte()`.

Fase 7: Free Shop (Opcional)

Si el pasajero tiene tiempo (`comprar == true`), intenta ingresar al FreeShop con un timeout. Si entra, puede comprar algo: setea `pagoPendiente=true`, el Cajero lo procesa (3 segundos), y el pasajero paga. Luego sale del FreeShop en orden FIFO usando el patrón de turnos (`turnoActual/esperandoTurno`).

Fase 8: Sala de Embarque

El pasajero llama a `sala.esperarLlamado()`. Adquiere el lock, guarda su grupo actual, incrementa `pasajerosEsperando` y espera en `while(grupo == miGrupo) llamado.await()`. El `EmpleadoSalon` detecta que hay pasajeros y llama a `llamarAEmbarcar()`, que incrementa el grupo y despierta a todos los pasajeros del ciclo actual.

Fase 9: Despegue del Avión

Después de embarcar, el pasajero ejecuta `aeropuerto.avionDespega.countDown()`. Cuando el último pasajero hace `countDown()`, el `CountDownLatch` llega a 0 y el hilo `Vuelo` se despierta imprimiendo: "EL AVION DESPEGA CON TODOS LOS PASAJEROS A BORDO".

7. Manejo de Casos Extremos

7.1. Cierre del Aeropuerto

Después de 30 segundos, `cerrarAeropuerto()` setea `abierto=false` y llama a `notificarAeropuertoCerrado()`. Este metodo:

- Adquiere `mutexAeropuerto` (Semaforo binario) para proteger la escritura.
- Setea `aeropuertoCerrado = true` dentro del `mutex`.
- Ejecuta `barrera.reset()` para romper la barrera.
- Los pasajeros que están en `barrera.await()` reciben `BrokenBarrierException`.
- Si el viaje es parcial (`subidosAhora % capacidad != 0`) y el aeropuerto está cerrado, un pasajero detecta la condición y usa `intentarConductor()` (protegido por `mutexArranque`) para lanzar un Conductor de forma segura.

7.2. Viajes Parciales

Cuando el aeropuerto cierra, puede haber menos pasajeros esperando en el transporte. En este caso:

- La barrera se resetea por `notificarAeropuertoCerrado()`.
- Los pasajeros reciben `BrokenBarrierException`.
- El primer pasajero que detecta la condición ejecuta `intentarConductor()`, protegido por el semaforo binario `mutexArranque`, para lanzar un `Conductor` de forma segura.
- Los demás pasajeros ven que `conductorArrancando` ya es `true` y esperan en `inicioUltimoViaje.acquire()`.
- Cuando el `Conductor` termina, `terminoRecorrido()` resetea `conductorArrancando` a `false` (bajo `mutexArranque`) y libera semaforos `puedeAbordar` e `inicioUltimoViaje`.

7.3. Free Shop Lleno

Si el `FreeShop` está lleno (`contador >= capacidadMax`), el pasajero espera con un `timeout` mediante `wait(restante)` dentro del bloque `synchronized`. Si no se libera un lugar a tiempo, el pasajero abandona el intento y va directamente a la sala de embarque. Esto previene que los pasajeros se queden bloqueados indefinidamente.

7.4. Pasajeros que No Pueden Ingresar

Si un pasajero llega después de que el aeropuerto cierra, `control.entrarAlAeropuerto()` retorna `false` (`abierto == false`) y el pasajero imprime un mensaje indicando que no pudo ingresar.

8. Conclusiones

El sistema implementado demuestra la correcta aplicación de los mecanismos de sincronización proveídos por Java para resolver un problema de concurrencia complejo. Cada mecanismo fue seleccionado según su afinidad para el problema específico:

- `TransporteATerminal` usa exclusivamente semáforos (junto con `CyclicBarrier`, `BlockingQueue` y `CountDownLatch`) para el control de capacidad, `mutex`, aparición del conductor y cierre del aeropuerto.
- `FreeShop` y `PuestoAtencion` usan exclusivamente monitores (`synchronized` + `wait/notifyAll`) para el control de capacidad, realizar pago y salida FIFO.
- `ControlAeropuerto`, `PuestoInformes` y `SalaEmbarque` usan exclusivamente `ReentrantLock` con `Condition` variables para la sincronización con múltiples conjuntos de espera.
- El `CountDownLatch` se utiliza para representar el despegue del avión (evento irreversible) y para detectar viajes parciales.
- El `CyclicBarrier` modela naturalmente la condición de "el transporte parte cuando se llena".
- El `Exchanger` facilita el intercambio simétrico de datos entre pasajero y empleado.
- El `BlockingQueue` proporciona control de capacidad acotada con bloqueo automático para los asientos del transporte.

El programa fue verificado ejecutándose múltiples veces con 20 pasajeros, confirmando que todos completan su recorrido y suben al avión, y que el mensaje de despegue aparece al final.